

STAT 240

Week 9: Clustering and Strings

Owen G. Ward

Today

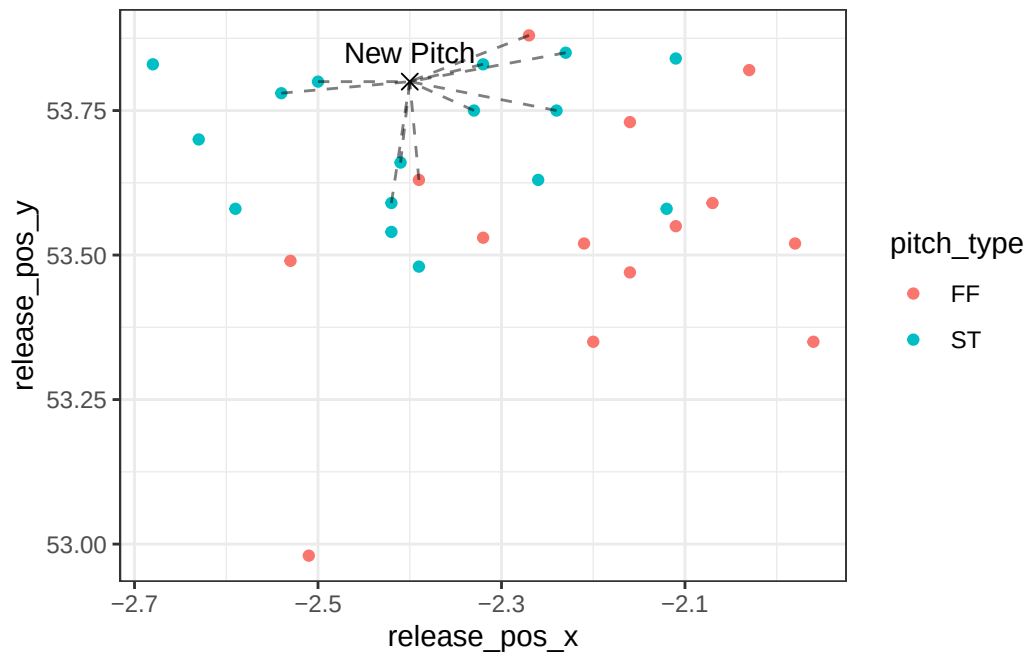
- Recap on classification and CV
- An introduction to clustering
- Manipulating Text Data: Strings

Recap: Classification

Predicting Categories

- We had numeric data X and we wanted to use it to predict a categorical variable Y
- Need different models to those for predicting numeric Y
- Need a different way to evaluate these models (RMSPE doesn't make sense)

What this looks like



KNN for Classification

- Find the nearest points in the training set to the X we want to predict at
- See which class occurs the most often in those nearby points
- This is our prediction for a new point at X

Finding Nearest Neighbours

- Use Euclidean distance to determine which points are nearest
- Can compute this using

$$\text{distance} = \sqrt{(x - x_{new})^2 + (y - y_{new})^2}$$

Evaluating Predictions

- Need something that can measure the difference between predictions of categories

$$\text{accuracy} = \frac{\text{how many correct predictions}}{\text{how many data points we predict at}}$$

- Can either predict the correct category or get it wrong, no other possibilities

Choosing K - Cross Validation

- To choose the optimal K in KNN we can only use the training data
- Not allowed access to the test set, otherwise would overfit
- Solve this using **Cross Validation**

5 Fold CV

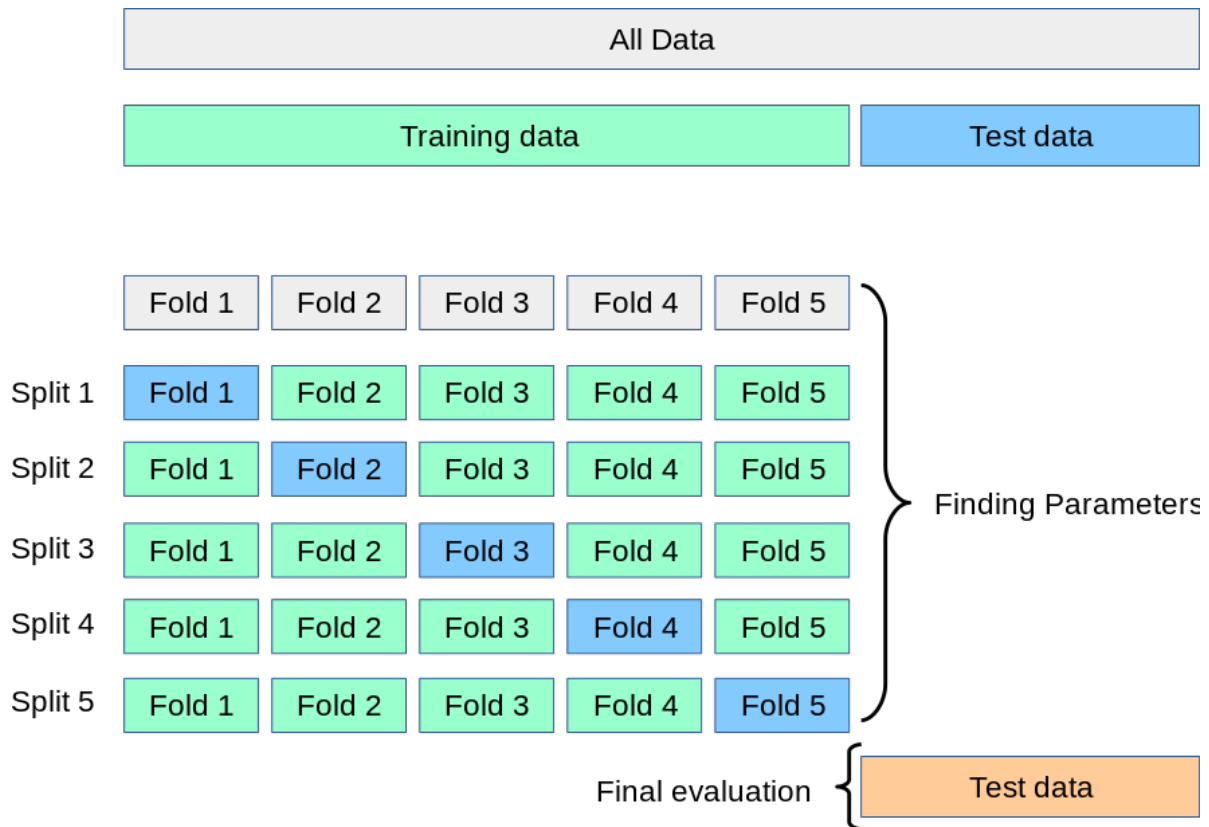


Figure 1: 5 Fold Cross Validation

The full process

```
library(class)
K_vals <- seq(from = 1, to = 200, by = 1)
K_acc <- rep(NA, length(K_vals))
set.seed(100)

CV_folds <- sample(1:5, size = nrow(training_data), replace = TRUE)
num_folds <- 5

for(j in 1:length(K_vals)) {
  k <- K_vals[j]
  CV_acc <- rep(NA, num_folds)
```

```

for(i in 1:num_folds){
  cv_train <- training_data[CV_folds != i, ]
  cv_val <- training_data[CV_folds == i, ]
  fit_knn <- knn(train = as.matrix(cbind(cv_train$release_pos_x,
                                         cv_train$release_pos_y)),
                 test = as.matrix(cbind(cv_val$release_pos_x,
                                         cv_val$release_pos_y)),
                 cl = cv_train$pitch_type, k = k)
  CV_acc[i] <- mean(fit_knn == cv_val$pitch_type)
}
K_acc[j] <- mean(CV_acc)
}

```

Clustering

What is Clustering?

- Group similar data items together
- Exploratory data analysis / pattern recognition
- Example of *unsupervised learning* (a branch of ML)

Unsupervised Learning

- A different data science approach to regression/classification
- No variable Y we are trying to predict
- Instead just want to identify similar types of data
 - Similar customers on some shopping website, etc
- Important data science tool for understanding the data, structure of the data

Unsupervised Learning

- No Y we are trying to predict!
- No way to “predict” or “evaluate” these methods, but give you information about the data
- Often use these alongside tools for prediction, for slightly different datasets

Simulated data

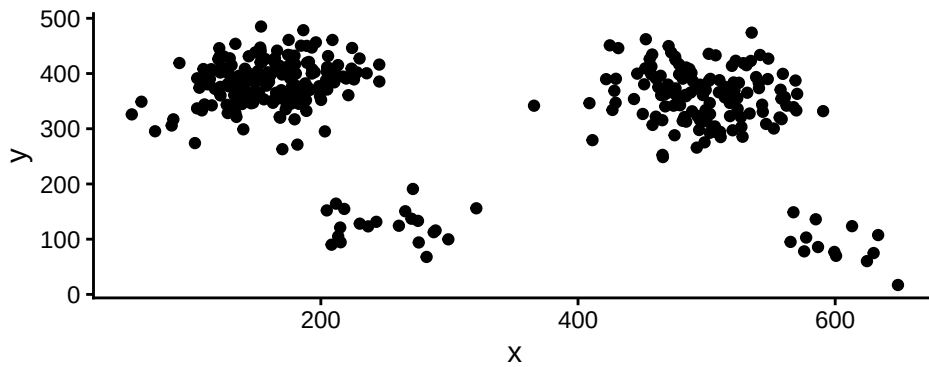
- Used [Calm Code](#) to simulate a dataset demonstrating clustering

reset copy json copy csv download json download csv ☒ A ☐ B ☐ C ☐ D



Simulated data

```
data <- read.csv("data.csv")
library(ggplot2)
ggplot(data, aes(x = x, y = y)) +
  theme_classic() +
  geom_point()
```



k-means clustering

The simplest clustering method is k-means clustering

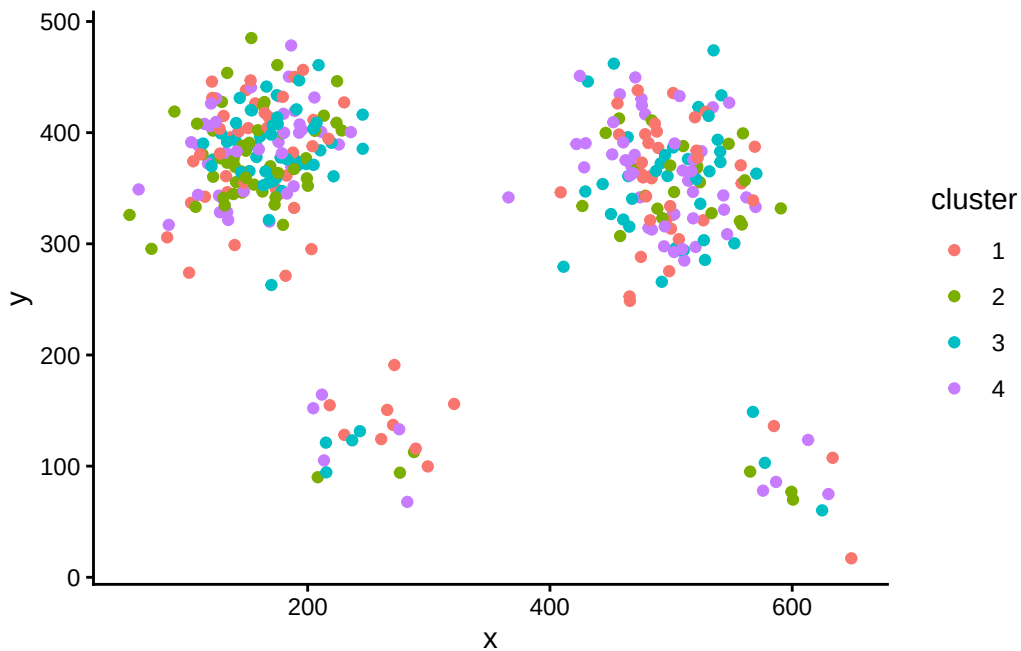
- Iterative method with random initialization
- The parameter is fixed: the number of clusters k
- Somewhat different to KNN!
 - There are ways to choose k but not using CV (no predictions)
 - Come back to this in a few minutes

k-means clustering in 2D

- Step 1: Randomly assign each data item to one of the k clusters

```
N <- dim(data)[1]
K <- 4
set.seed(240)
cluster <- sample(K, size = N, replace = TRUE)
data$cluster <- cluster
p <- ggplot(data, aes(x = x, y = y, color = as.factor(cluster))) +
  labs(color = "cluster") +
  theme_classic() +
  geom_point()
print(p)
```

Random Clusters

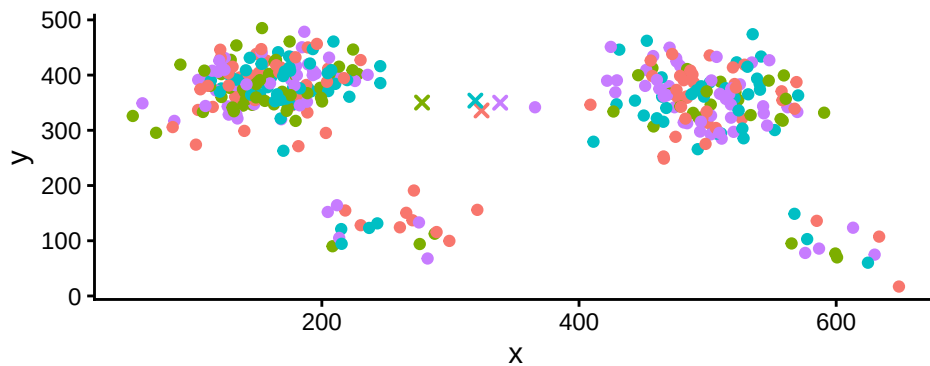


- Step 2: Compute the mean x and y coordinate of each cluster (this is called a centroid)
- If a cluster is empty, assign the corresponding centroid to a randomly selected data item

```
mus <- matrix(NA, K, 3)
colnames(mus) <- c("x", "y", "cluster")
for (k in 1:K) {
  mus[k, 1] <- mean(data$x[data$cluster == k])
  mus[k, 2] <- mean(data$y[data$cluster == k])
  mus[k, 3] <- k
}
q <- p + geom_point(as.data.frame(mus),
  mapping = aes(x = x, y = y,
    color = as.factor(cluster)),
  shape = 4, stroke = 1) +
  theme(legend.position = "none")
print(q)
```

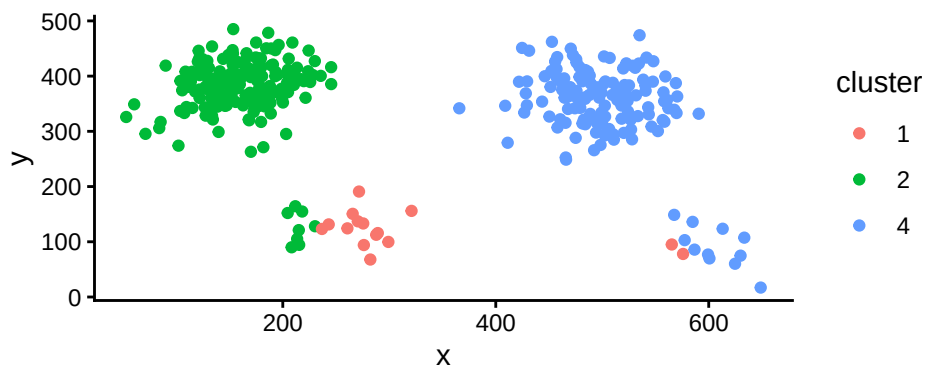
k-means clustering in 2D

- Step 2:



k-means clustering in 2D

- Step 3: Reassign each data item to the cluster with the closest centroid



Notes

- This only uses 3 clusters here, but still have a 4th cluster
- If a cluster is empty, assign the corresponding centroid to a randomly selected data item
- So will randomly take one of these points and make that cluster 3 before the next step

k-means clustering in 2D

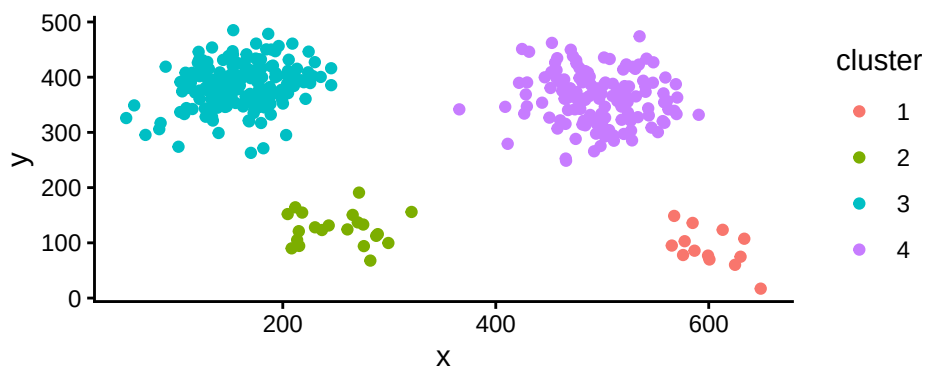
- Iterate by repeating steps 2 and 3 until a stopping condition is met
- Examples of stopping conditions:
 1. A maximum number of iterations is reached
 2. The cluster assignment doesn't change
 3. The centroids move by only a small amount
- Stop the first time one of these happens

k-means clustering in 2D

- The result may look something like this:

```
set.seed(240)
result <- kmeans(data[, 1:2], 4, iter.max = 100)
data$cluster <- result$cluster
p <- ggplot(data, aes(x = x, y = y, color = as.factor(cluster))) +
  labs(color = "cluster") +
  theme_classic() +
  geom_point()
print(p)
```

Final Clusters



The clusters

- The points in a cluster are more similar to each other than observations in a different group
- That's the underlying quantity clustering is capturing in the data, which is pretty vague!
- Won't get into the math behind this, will see it in other classes

k-means theory

- The k-means algorithm is one of the simplest clustering algorithms
- A big drawback is that you have to already know how many clusters you want
- Another drawback is that the “variance” of each cluster is the same: A cluster that is very compact might have a k-means solution that includes aspects of other clusters

k-means theory

- Another drawback is that k-means clustering doesn't work for “non convex” clusters
- No clear rules for how to implement this, involves quite a bit of trial and error!

k-means can get stuck

- When you run `kmeans` you randomly put each point in one of the k clusters
- How you do this can impact the clustering you get
- To get around this:
 - Start from multiple different random clusters (`nstart` in R)
 - Choose the one which gives the best separated clusters

Implementing this in R

- Use the `kmeans` function, don't need any packages
- Need the data to all be numeric
- Specify k and how many iterations to run (stopping rule)

```
result <- kmeans(data[, 1:2], 4, iter.max = 100)
head(result$cluster) ## cluster each point is in
```

```
[1] 4 4 4 4 1 1
```

-
- More random starts could give different clusters

```
result2 <- kmeans(data[, 1:2], 4, iter.max = 100, nstart = 10)
head(result2$cluster)
```

```
[1] 1 1 1 1 1 1
```

- The numbers can change from run to run, same numbers in one fit mean the same cluster

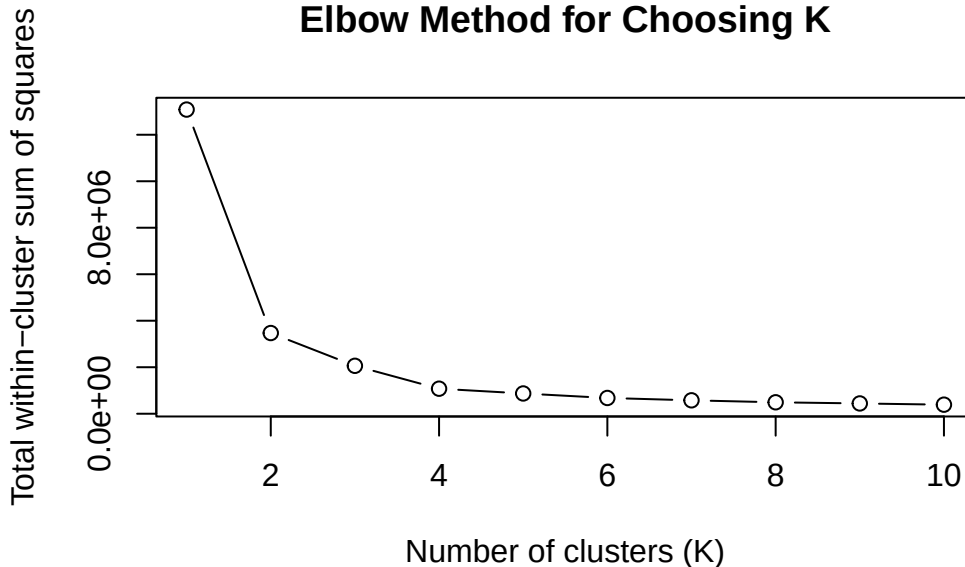
Choosing K

- For different values of K , compute the variance of points within the clusters
- Plot these, choose the value of K where this stops decreasing

```
K_vals <- 1:10
wss <- rep(NA, length(K_vals))

for (i in seq_along(K_vals)) {
  result <- kmeans(data[, 1:2], centers = K_vals[i], iter.max = 100, nstart = 10)
  wss[i] <- result$tot.withinss
}
```

Plotting this



- Find the “elbow”, where this quantity stops decreasing, maybe $K = 3/4$ here, not very exact!

Exercise

- Suppose we have the dataset `peng_scaled` with 2 variables and we have the positions of 3 centroids, `centroids`. Write an R function `my_dist` which will compute the distance between these centroids and all points in the dataset. If you have n rows in your data and K centroids, this will be an $n \times K$ matrix, where the entries in the first row will be the distance from that point to each centroid, respectively.

Attaching package: 'palmerpenguins'

The following objects are masked from 'package:datasets':

`penguins`, `penguins_raw`

- To create a matrix `dist_mat <- matrix(NA, nrow = ..., ncol = ...)`

```
peng_scaled <- read.csv("peng_scaled.csv")
```

```
set.seed(100)
head(peng_scaled)
```

```
# A tibble: 6 x 2
  flipper_scaled bill_scaled
      <dbl>         <dbl>
1      -1.42        -0.883
2      -1.06        -0.810
3      -0.421       -0.663
4      -0.563       -1.32
5      -0.776       -0.847
6      -1.42        -0.920
```

```
centroids <- sample_n(peng_scaled, size = 3)
centroids
```

```
# A tibble: 3 x 2
  flipper_scaled bill_scaled
      <dbl>         <dbl>
1      0.646         0.491
2     -0.563        -0.773
3      1.14          0.472
```

Solution

Text Data: Strings

String manipulation

String manipulation for text data comes up all the time!

- Natural language processing (NLP)
 - Sentiment analysis
 - Chatbots, translation
 - Electronic healthcare records ...

-
- Preprocessing data sources
 - Normalization of factors
 - Parsing websites or documents ...
 - Data analysis
 - Understanding DNA ...

String manipulation

- We'll review string manipulation in base R.
- First, we will read the text of the Great Gatsby (Fitzgerald 1925) into a single string variable

```
fn <- "gatsby.txt"
s <- readChar(fn, file.info(fn)$size)
nchar(s) # Print number of characters in text
```

```
[1] 296673
```

```
length(s)
```

```
[1] 1
```

- This reads the whole text into a single character string

Why we need to manipulate this

```
substring(s, 3504, 3775)
```

```
[1] " If personality is an unbroken series of\r\nsuccessful gestures, then there was something
```

- This is not just the text, other characters that we need to deal with
- Probably don't care about punctuation also

String manipulation

- We want a character vector with one word per element

```
x <- strsplit(s, "\\s+")
length(x)
```

```
[1] 1
```

- Here, "\\s+" is telling R to split based on **whitespace** in the character string
- This is what's known as a **regular expression**, way to detect common patterns in text
- Will come back to these in more detail next week
- Places this output in a list of length 1

```
x <- unlist(x)
print(x[507:517])
```

```
[1] "I"      "felt"   "that"   "I"      "wanted" "the"    "world"
[8] "to"     "be"     "in"     "uniform"
```

- Here because we only have one element in our list, can just use **unlist** to basically unnest this into a vector

```
length(x)
```

```
[1] 51257
```

Wordclouds

- We can create a wordcloud of the document (visualization, not much use for actual data science)

```
library(wordcloud)
word_counts <- table(x)
wordcloud(names(word_counts), word_counts) ## this is very slow
```

- `word_counts` will store how many times each word occurs in the document

```
Loading required package: RColorBrewer
```

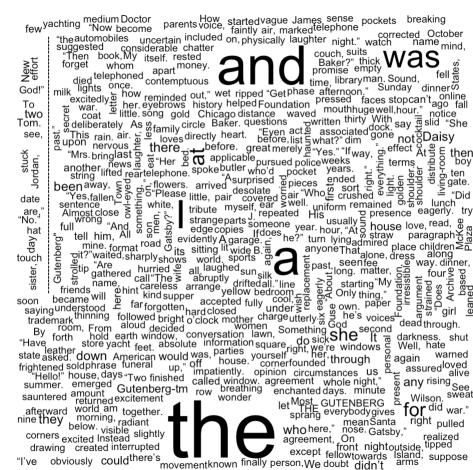
```
word_counts[101:107]
```

x

"crazy	"Crazy	"creative	"Daisy	"Daisy,	"Daisy!	"Daisy?"
1	2	1	1	1	1	1

- You may want to process these a bit more!

A strange wordcloud



Wordclouds

- In NLP, we often ignore “stopwords” such as “a” and “the”. Here, we also ignore infrequent words

```
library(stopwords)
stopwords("en")[1:10]
```

```
[1] "i"          "me"         "my"         "myself"     "we"         "our"
[7] "ours"       "ourselves" "you"        "your"
```

```
x <- tolower(x) ## so Crazy and crazy the same
x <- x[!(x %in% stopwords("en"))]
word_counts <- table(x)
word_counts <- word_counts[word_counts >= 20] ## remove rare words
word_counts[101:110]
```

x									
know	last	left	let	light	like	little	long	look	looked
70	41	37	25	25	116	91	55	34	82

A better wordcloud



Low level string manipulation

Easiest R package: `stringr`

- Match a string and extract it `str_extract`
- Detect a matching string `str_detect`
- Replace one string with another `str_replace`
- Split a string on a substring `str_split`
- Many other options possible!

Strings: extracting

```
library(stringr)
a <- c("apples x4", "bag of flour", "bag of sugar", "milk x2")
str_extract(a, "\\d") ## looking for digits
```

```
[1] "4" NA  NA  "2"
```

```
str_extract(a, "[a-z]+") ## anything starting with one or more letters
```

```
[1] "apples" "bag"    "bag"    "milk"
```

```
str_extract_all(a, "[a-z]+") ## anything starting with one or more letters
```

```
[[1]]
[1] "apples" "x"
```

```
[[2]]
[1] "bag"    "of"     "flour"
```

```
[[3]]
[1] "bag"    "of"     "sugar"
```

```
[[4]]
[1] "milk" "x"
```

Strings: detecting

```
a <- c("apple", "banana", "pear", "pineapple")
str_detect(a, "a")
```

```
[1] TRUE TRUE TRUE TRUE
```

```
str_detect(a, "^a") ## starts with a
```

```
[1] TRUE FALSE FALSE FALSE
```

```
str_detect(a, "apple")
```

```
[1] TRUE FALSE FALSE TRUE
```

```
str_detect(a, "e$") ## ends in e
```

```
[1] TRUE FALSE FALSE TRUE
```

Strings: replacing

```
a <- c("one apple", "two pears", "three bananas")
str_replace(a, "[aeiou]", "-")
```

```
[1] "-ne apple"      "tw- pears"      "thr-e bananas"
```

```
str_replace_all(a, "[aeiou]", "-")
```

```
[1] "-n- -ppl-"      "tw- p--rs"      "thr-- b-n-n-s"
```

Strings: splitting

```
a <- c(
  "apples and oranges and pears and bananas",
  "pineapples and mangos and guavas"
)
str_split(a, " and ")
```

```
[[1]]
[1] "apples" "oranges" "pears" "bananas"
```

```
[[2]]
[1] "pineapples" "mangos" "guavas"
```

```
str_split(a, "\\s+")
```

```
[[1]]
[1] "apples" "and" "oranges" "and" "pears" "and" "bananas"
```

```
[[2]]
[1] "pineapples" "and" "mangos" "and" "guavas"
```

Exercise

- The following code gives you the text of the Austen novel Emma, with each line in a separate character vector.

```
library(janeaustenr)
data("emma")
emma[1:4]
```

```
[1] "EMMA" "" "By Jane Austen" ""
```

```
emma[106:108]
```

```
[1] "reconciled to his own daughter's marrying, nor could ever speak of her"
[2] "but with compassion, though it had been entirely a match of affection,"
[3] "when he was now obliged to part with Miss Taylor too; and from his"
```

- What proportion of these lines contain the word Emma? What about another more minor character, Jane?

Solution

More details

- What if we wanted to find how many times the name **Emma** used?
 - Will this give us the correct answer?
-

Sentiment Analysis

- Sentiment analysis a way to determine how positive/negative text is, how this changes over time
 - Number of positive words - number of negative words
- Interesting way to analyse text
- Anyone read any Jane Austen novels?
- See [the Tidy Text Mining book](#) for more interesting applications

Sentiments in Austen Novels

